R Code Examples from WDDA Lecture 6 (Chapter 6)

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-05-14

Contents

1 Introduction **1**

2 Preparation **2**

3 Investigating the Explanatory Variables **2**

4 Multiple Linear Regression (TV and Radio) **4**

5 Prediction with the Model **7**

6 Regression Surface (Plane of Best Fit) **8**

7 Goodness of Fit and Explanatory Power **9**

8 Residual Analysis **10**

9 Extension: Multiple Regression with Three Predictors **15**

10 Non-linear Models: Quadratic and Polynomial Regression **16**

11 Model Comparison: R^2, Adjusted R^2 and RSE **19**

12 Summary **21**

1 Introduction

This document collects all R code examples from the slides of WDDA FS2026 Lecture 6 (Chapter 6). The focus is on multiple linear regression, an extension of simple linear regression to several predictors.

Multiple linear regression is a statistical method that models the relationship between a dependent variable and several independent variables. In contrast to simple linear regression, which uses only a single predictor, multiple regression allows us to investigate more complex relationships.

The central concepts in this area are:

1. **Multiple regression with several predictors**: How do we model the influence of several variables?
2. **Residual analysis and goodness of fit**: How well does the model fit the data?
3. **Prediction and error margin**: How accurate are the model's predictions?
4. **Non-linear and polynomial regression**: How do we model non-linear relationships?
5. **Model comparison**: How do we compare different regression models?

2 Preparation

```
# Load packages
library(scatterplot3d)
library(rgl)
library(car)
library(readxl)

# Load data
data <- read_excel("data/WDDA_06.xlsx", sheet = "Advertising")

# Make variables directly addressable
attach(data)
```

```
## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
##  $ TV       : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
##  $ radio    : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
##  $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
##  $ sales    : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R explanation:

- The function `library()` loads an installed package into the current R session.
- The package `scatterplot3d` enables the creation of 3D scatterplots.
- The package `rgl` provides interactive 3D visualisations.
- The package `car` (Companion to Applied Regression) contains functions for regression analysis.
- With `read_excel()` from the package `readxl` you can import Excel files.
- The function `attach()` makes the variables of a data frame directly addressable, without having to prefix the data frame name.
- The function `str()` shows the structure of an object, including data types and dimensions.

Note on data paths: In R Markdown files, relative paths are interpreted from the location of the file, hence the path `"../data/WDDA_06.xlsx"`. In ordinary R scripts the path would be specified relative to the working directory.

3 Investigating the Explanatory Variables

Before we begin with the multiple regression, we examine the relationships between the variables.

```
# Correlations
cor_tv <- cor(sales, TV)
cor_radio <- cor(sales, radio)
cat("Correlation (TV vs. Sales):", round(cor_tv, 4), "\n")
```

```
## Correlation (TV vs. Sales): 0.7822
```

```
cat("Correlation (Radio vs. Sales):", round(cor_radio, 4), "\n\n")
```

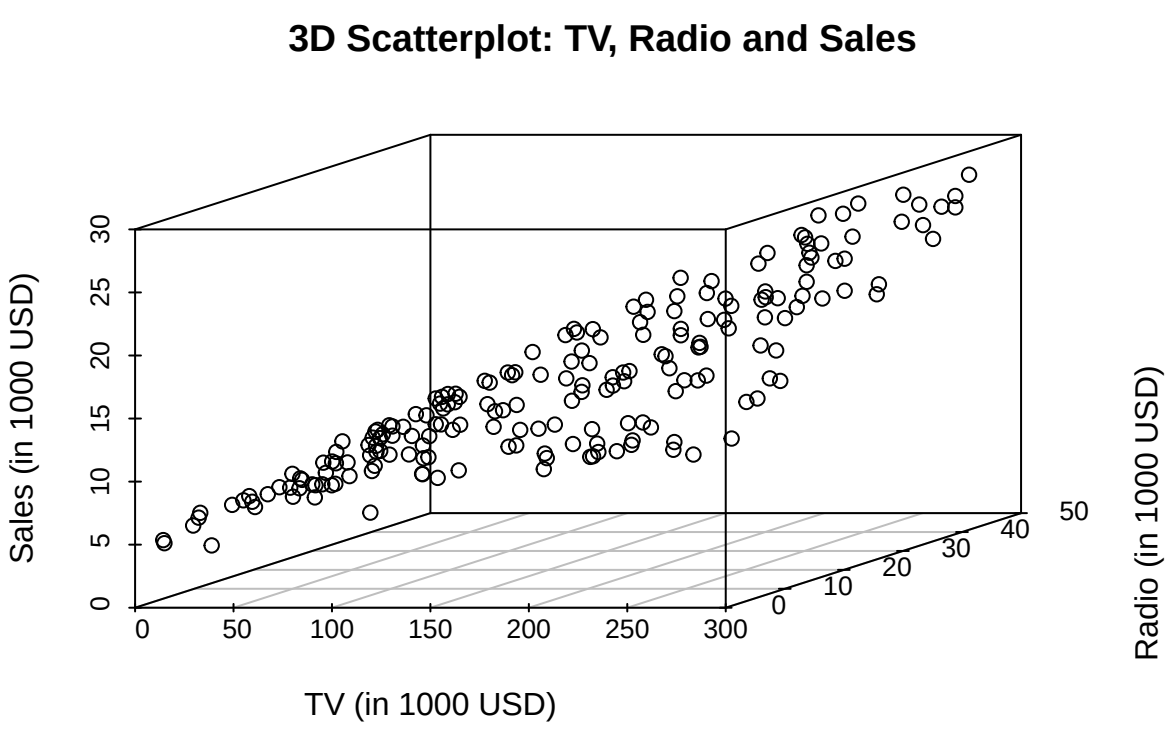
```
## Correlation (Radio vs. Sales): 0.5762
```

```
# Correlation matrix
cor_matrix <- cor(data[, c("TV", "radio", "newspaper", "sales")])
```

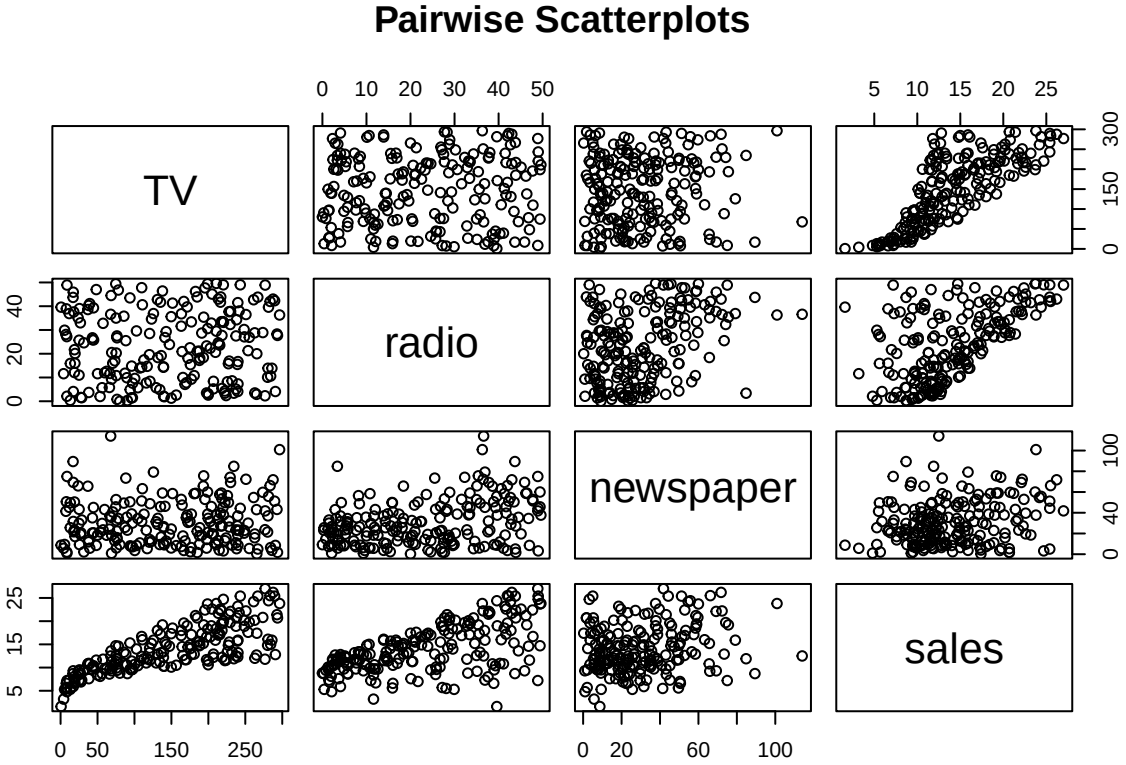
```r
print(round(cor_matrix, 4))
```

```
##                TV  radio newspaper  sales
## TV         1.0000 0.0548    0.0566 0.7822
## radio      0.0548 1.0000    0.3541 0.5762
## newspaper  0.0566 0.3541    1.0000 0.2283
## sales      0.7822 0.5762    0.2283 1.0000
```

```r
# 3D visualisation (TV, Radio, Sales)
scatterplot3d(x = TV, y = radio, z = sales, scale.y = 0.9, angle = 30,
              main = "3D Scatterplot: TV, Radio and Sales",
              xlab = "TV (in 1000 USD)", ylab = "Radio (in 1000 USD)", zlab = "Sales (in
               ↪  1000 USD)")
```




```r
# Pairwise scatterplots
pairs(data[, c("TV", "radio", "newspaper", "sales")],
      main = "Pairwise Scatterplots")
```




```
# Dynamic 3D scatterplot
# Note: this function opens an interactive window
scatter3d(x = TV, z = radio, y = sales, surface = FALSE)
```

Statistical explanation:

- The **correlation** is a measure of the linear relationship between two variables and lies between -1 and +1.
- A **correlation matrix** shows the pairwise correlations between several variables.
- **3D scatterplots** allow the visualisation of the relationship between three variables.
- **Pairwise scatterplots** show the relationships between all pairs of variables in a dataset.
- In multiple regression it is important to understand the relationships between the predictors, since multicollinearity (strong correlations between predictors) can complicate the interpretation of the coefficients.

R explanation:

- The function `cor()` computes the Pearson correlation between two vectors or a correlation matrix for a data frame.
- The function `scatterplot3d()` from the package of the same name creates a 3D scatterplot.
- The function `scatter3d()` from the package `car` creates an interactive 3D scatterplot.
- The function `pairs()` creates a matrix of pairwise scatterplots.

4 Multiple Linear Regression (TV and Radio)

We now run a multiple regression with two predictors: TV and Radio.

```
# Model with two predictors
md2 <- lm(sales ~ TV + radio)
```

```r
summary(md2)
```

```
##
## Call:
## lm(formula = sales ~ TV + radio)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.7977 -0.8752  0.2422  1.1708  2.8328
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.92110    0.29449   9.919   <2e-16 ***
## TV           0.04575    0.00139  32.909   <2e-16 ***
## radio        0.18799    0.00804  23.382   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.681 on 197 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8962
## F-statistic: 859.6 on 2 and 197 DF,  p-value: < 2.2e-16
```

```r
# Regression equation
B <- coef(md2)
cat("Regression equation:\n")
```

```
## Regression equation:
```

```r
cat("sales  ", round(B[1], 3), "+", round(B[2], 3), "* TV +", round(B[3], 3), "*
 ↪  radio\n\n")
```

```
## sales   2.921 + 0.046 * TV + 0.188 * radio
```

```r
# Comparison with simple regression (TV only)
md1 <- lm(sales ~ TV)
summary(md1)
```

```
##
## Call:
## lm(formula = sales ~ TV)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 7.032594   0.457843   15.36   <2e-16 ***
## TV          0.047537   0.002691   17.67   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 3.259 on 198 degrees of freedom
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
## F-statistic: 312.1 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
# Comparison of the coefficients
cat("Coefficient for TV (simple regression):", round(coef(md1)[2], 4), "\n")
```

```
## Coefficient for TV (simple regression): 0.0475
```

```
cat("Coefficient for TV (multiple regression):", round(coef(md2)[2], 4), "\n\n")
```

```
## Coefficient for TV (multiple regression): 0.0458
```

```
# Interpretation of the coefficients
cat("Interpretation of the coefficients (multiple regression):\n")
```

```
## Interpretation of the coefficients (multiple regression):
```

```
cat("- Intercept:", round(B[1], 3),
    "→ expected sales when TV = 0 and radio = 0 (in thousands of USD)\n")
```

```
## - Intercept: 2.921 → expected sales when TV = 0 and radio = 0 (in thousands of USD)
```

```
cat("- TV coefficient:", round(B[2], 4),
    "→ expected increase in sales for +1 unit of TV, holding radio constant\n")
```

```
## - TV coefficient: 0.0458 → expected increase in sales for +1 unit of TV, holding radio constant
```

```
cat("- Radio coefficient:", round(B[3], 4),
    "→ expected increase in sales for +1 unit of radio, holding TV constant\n")
```

```
## - Radio coefficient: 0.188 → expected increase in sales for +1 unit of radio, holding TV constant
```

Statistical explanation:

- **Multiple linear regression** models the relationship between a dependent variable and several independent variables: y = + x + x + … + x +
- The **regression coefficients** (, , …) give the influence of each independent variable on the dependent variable when all other variables are held constant.
- The **intercept** ( ) is the expected value of the dependent variable when all independent variables are zero.
- The **interpretation of the coefficients** in multiple regression differs from that in simple regression:
  - In simple regression, the coefficient gives the total effect of the variable.
  - In multiple regression, the coefficient gives the partial effect of the variable when all other variables are held constant.
- The coefficients can differ between simple and multiple regression when the predictors are correlated.

R explanation:

- The function `lm()` (linear model) fits a linear regression model to the data.
- The formula `sales ~ TV + radio` in `lm()` specifies that `sales` is the dependent variable and `TV` and `radio` are the independent variables.
- The function `summary()` provides a detailed summary of the regression model, including coefficients, standard errors, t-values, p-values and R^2.

- The function `coef()` extracts the coefficients from a regression model.

5 Prediction with the Model

With the regression model we can make predictions for new values of the independent variables.

```
# Prediction with error margin
RSE <- summary(md2)$sigma
tv_new <- 230.1
radio_new <- 37.8
predicted <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new))
pred_interval <- predicted + c(-1, 1) * 2 * RSE
cat("Forecast for TV =", tv_new, "and Radio =", radio_new, ":", round(predicted, 2),
 ↪  "\n")
```

```
## Forecast for TV = 230.1 and Radio = 37.8 : 20.56
```

```
cat("95% prediction interval:", round(pred_interval[1], 2), "to", round(pred_interval[2],
 ↪  2), "\n\n")
```

```
## 95% prediction interval: 17.19 to 23.92
```

```
# Exact prediction interval
pred_interval_exact <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new),
                               interval = "prediction", level = 0.95)
cat("Exact 95% prediction interval:",
    round(pred_interval_exact[2], 2), "to",
    round(pred_interval_exact[3], 2), "\n")
```

```
## Exact 95% prediction interval: 17.22 to 23.89
```

```
# Confidence interval for the expected value
conf_interval <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new),
                         interval = "confidence", level = 0.95)
cat("95% confidence interval for the expected value:",
    round(conf_interval[2], 2), "to",
    round(conf_interval[3], 2), "\n")
```

```
## 95% confidence interval for the expected value: 20.16 to 20.95
```

Statistical explanation:

- The **prediction** (forecast) is the predicted value of the dependent variable for given values of the independent variables: $\hat{y} =$ + x + x + … + x
- The **residual standard error (RSE)** is a measure of the typical size of the residuals and has the same unit as the dependent variable.
- The **prediction interval** gives the range in which a single new value will lie with a given probability.
- The **confidence interval for the expected value** gives the range in which the true expected value lies with a given probability.
- Prediction intervals are wider than confidence intervals for the expected value, because they additionally take into account the variation of individual observations around the expected value.

R explanation:

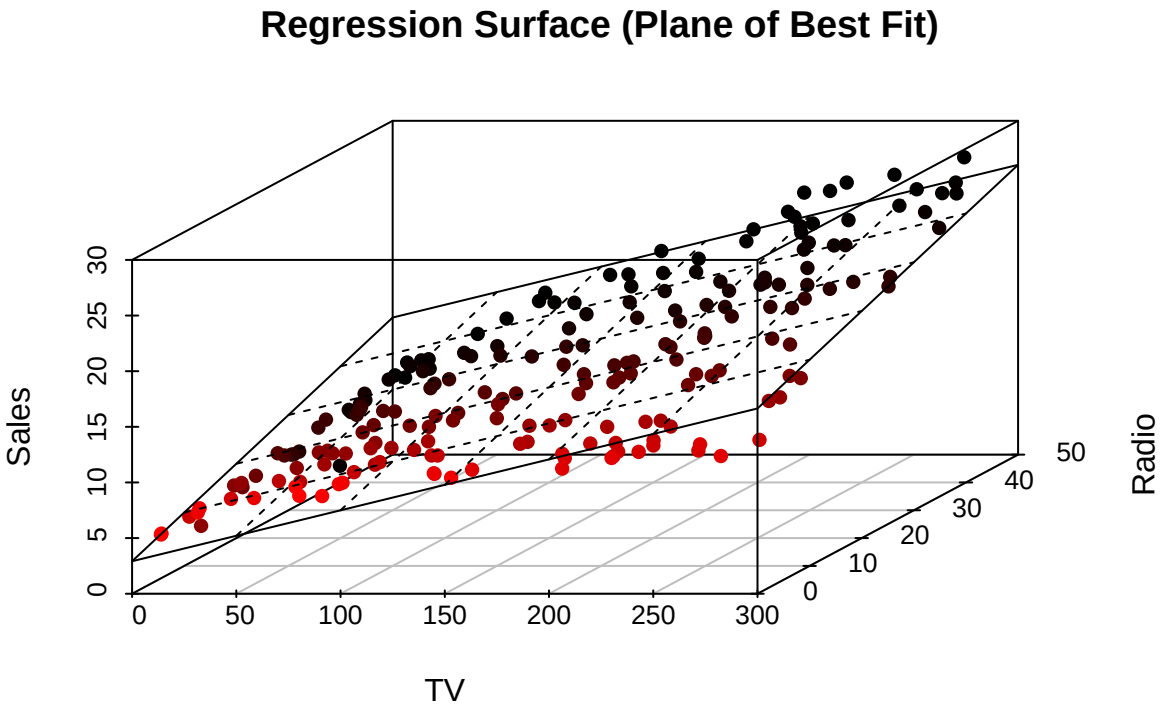
- The function `predict()` computes predicted values for a regression model.
- The parameter `newdata` in `predict()` specifies a data frame with the values of the predictors for which predictions are to be made.
- The parameter `interval` in `predict()` specifies whether confidence or prediction intervals are to be computed.
- The parameter `level` in `predict()` specifies the confidence level.
- The expression `summary(md2)$sigma` extracts the RSE from the regression object.

6 Regression Surface (Plane of Best Fit)

In multiple regression with two predictors we can visualise the regression surface in 3D space.

```
# Visualisation of the regression surface
# Note: this function opens an interactive window
scatter3d(x = TV, z = radio, y = sales, fit = "linear", residuals = TRUE, grid = TRUE)
```

```
# Static visualisation of the regression surface
s3d <- scatterplot3d(TV, radio, sales,
                     main = "Regression Surface (Plane of Best Fit)",
                     xlab = "TV", ylab = "Radio", zlab = "Sales",
                     pch = 16, highlight.3d = TRUE, angle = 45)

# Add the regression surface
my.lm <- lm(sales ~ TV + radio)
s3d$plane3d(my.lm, lty.box = "solid")
```



Statistical explanation:

- In **multiple regression with two predictors**, the relationship is represented by a plane in three-

dimensional space: z = + x + y
- This plane is called the **regression surface** (plane of best fit).
- The regression surface is chosen so that the sum of the squared vertical distances of the data points to the surface is minimised.
- The **residuals** are the vertical distances of the data points to the regression surface.
- Visualising the regression surface helps to understand the joint influence of the two predictors on the dependent variable.

R explanation:

- The function `scatter3d()` from the package `car` creates an interactive 3D scatterplot with a regression surface.
- The parameter `fit = "linear"` in `scatter3d()` specifies that a linear regression surface should be fitted.
- The parameter `residuals = TRUE` in `scatter3d()` displays the residuals as vertical lines.
- The function `scatterplot3d()` creates a static 3D scatterplot.
- The method `plane3d()` adds a regression surface to a `scatterplot3d` object.

7 Goodness of Fit and Explanatory Power

We now examine how well the model explains the variation in the data.

```
# Calculation of TSS, RSS and R²
TSS <- sum((sales - mean(sales))^2)
RSS <- sum((sales - predict(md2))^2)
r_squared <- 1 - (RSS / TSS)

cat("Coefficient of determination (R²):", round(r_squared, 4), "\n")
```

```
## Coefficient of determination (R²): 0.8972
```

```
cat("TV and Radio explain", round(r_squared * 100, 2), "% of the variation in
 ↪  Sales.\n\n")
```

```
## TV and Radio explain 89.72 % of the variation in Sales.
```

```
# Comparison with R² from summary()
r_squared_summary <- summary(md2)$r.squared
cat("R² from summary():", round(r_squared_summary, 4), "\n")
```

```
## R² from summary(): 0.8972
```

```
# Comparison with simple regression (TV only)
r_squared_tv <- summary(md1)$r.squared
cat("R² (TV only):", round(r_squared_tv, 4), "\n")
```

```
## R² (TV only): 0.6119
```

```
cat("R² (TV + Radio):", round(r_squared_summary, 4), "\n")
```

```
## R² (TV + Radio): 0.8972
```

```r
cat("Increase in R² from adding Radio:",
    round((r_squared_summary - r_squared_tv) * 100, 2), "percentage points\n")
```

```
## Increase in R² from adding Radio: 28.53 percentage points
```

Statistical explanation:

- The **coefficient of determination R^2** gives the proportion of the variation in the dependent variable that is explained by the regression model.
- R^2 lies between 0 and 1:
  - $R^2 = 0$: the model explains none of the variation in the data.
  - $R^2 = 1$: the model explains all of the variation in the data.
- R^2 can be computed in several ways:
  - R^2 = 1 - (RSS / TSS)
  - R^2 = ESS / TSS
- A higher R^2 means a better fit of the model to the data.
- Adding further predictors can only increase R^2 or leave it unchanged, never decrease it.
- This can lead to overfitting if too many predictors are added.

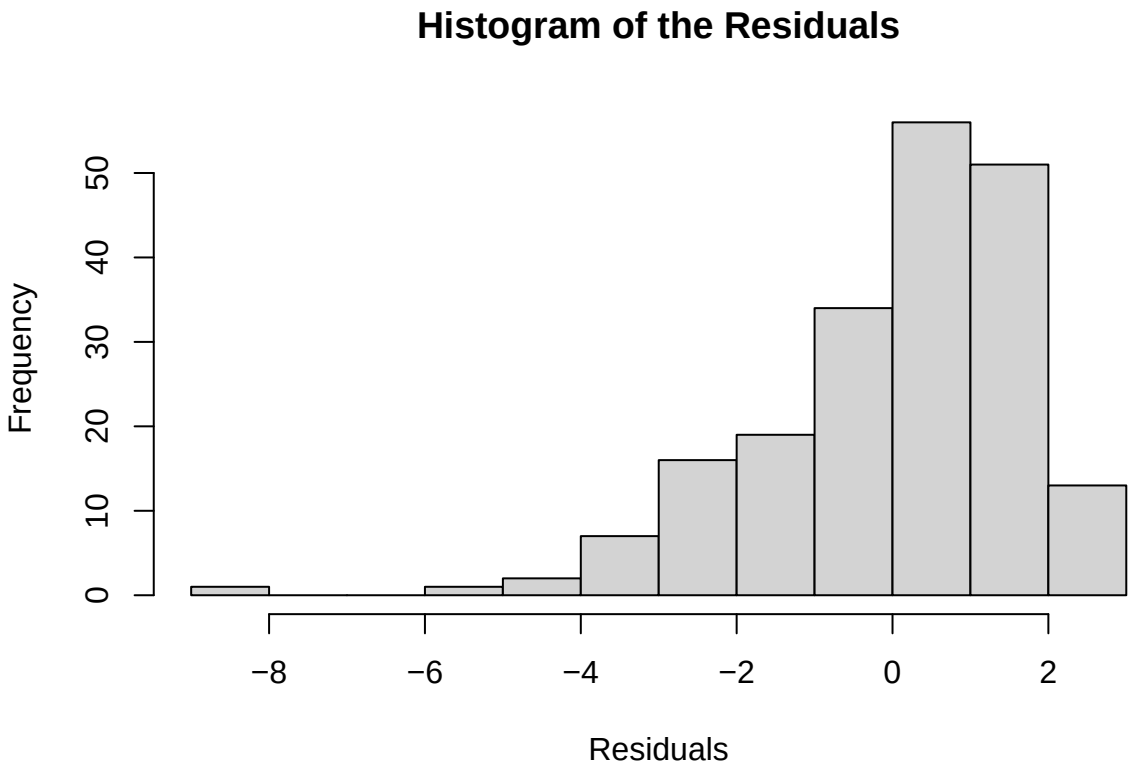
R explanation:

- The function `predict()` without further parameters computes the predicted values for the observations in the training dataset.
- The expression `summary(md2)$r.squared` extracts the R^2 value from the regression object.
- R^2 is computed directly via the formula R^2 = 1 - (RSS / TSS).

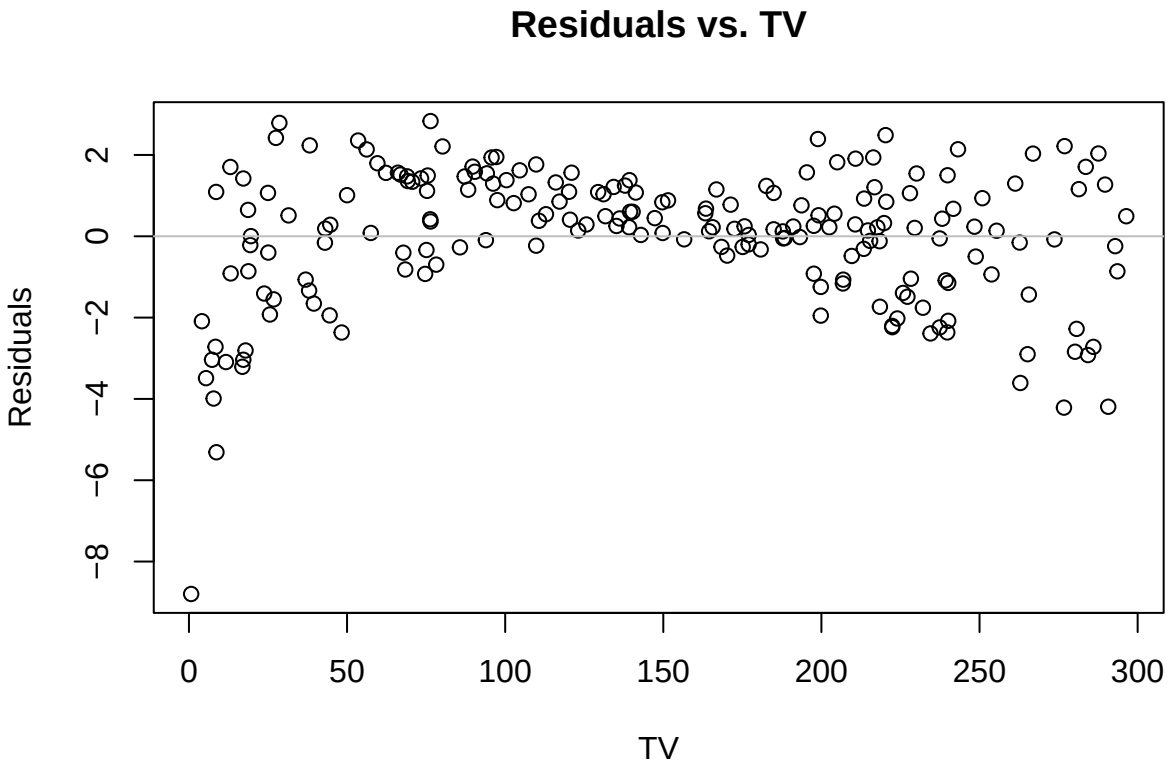
8 Residual Analysis

The analysis of the residuals is an important step for checking the assumptions of linear regression.

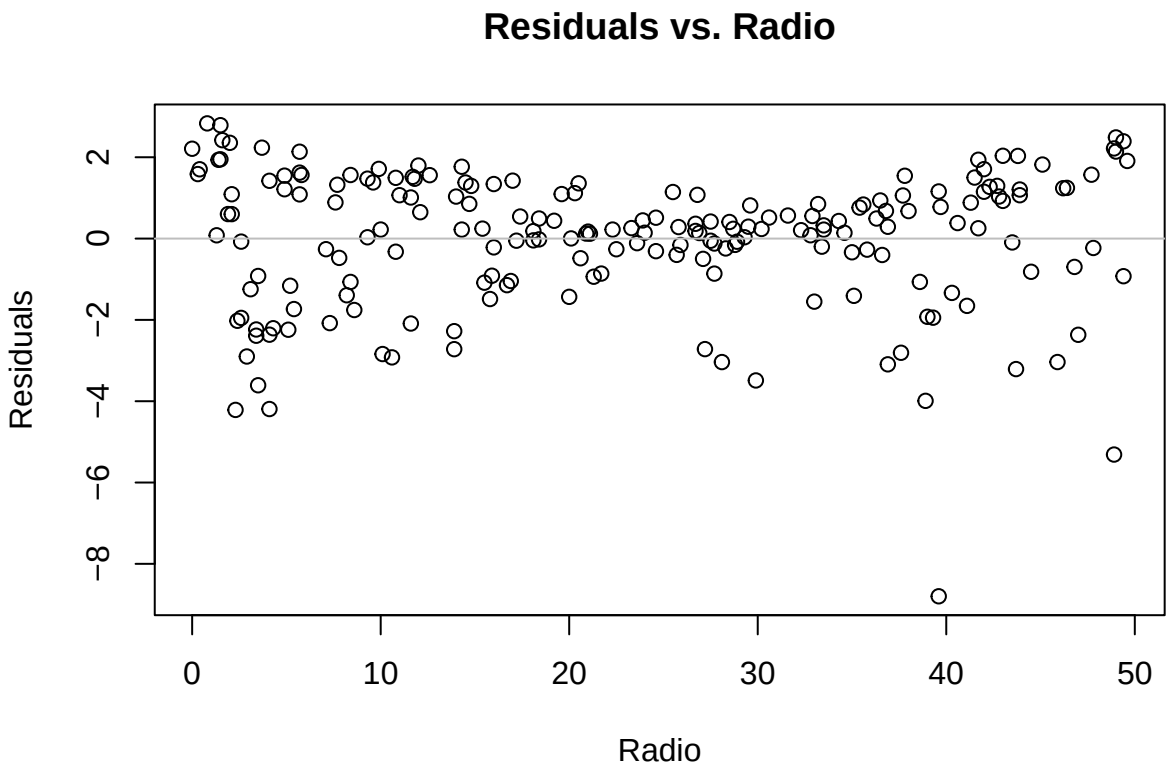
```r
# Residual plots
hist(resid(md2), main = "Histogram of the Residuals", xlab = "Residuals")
```



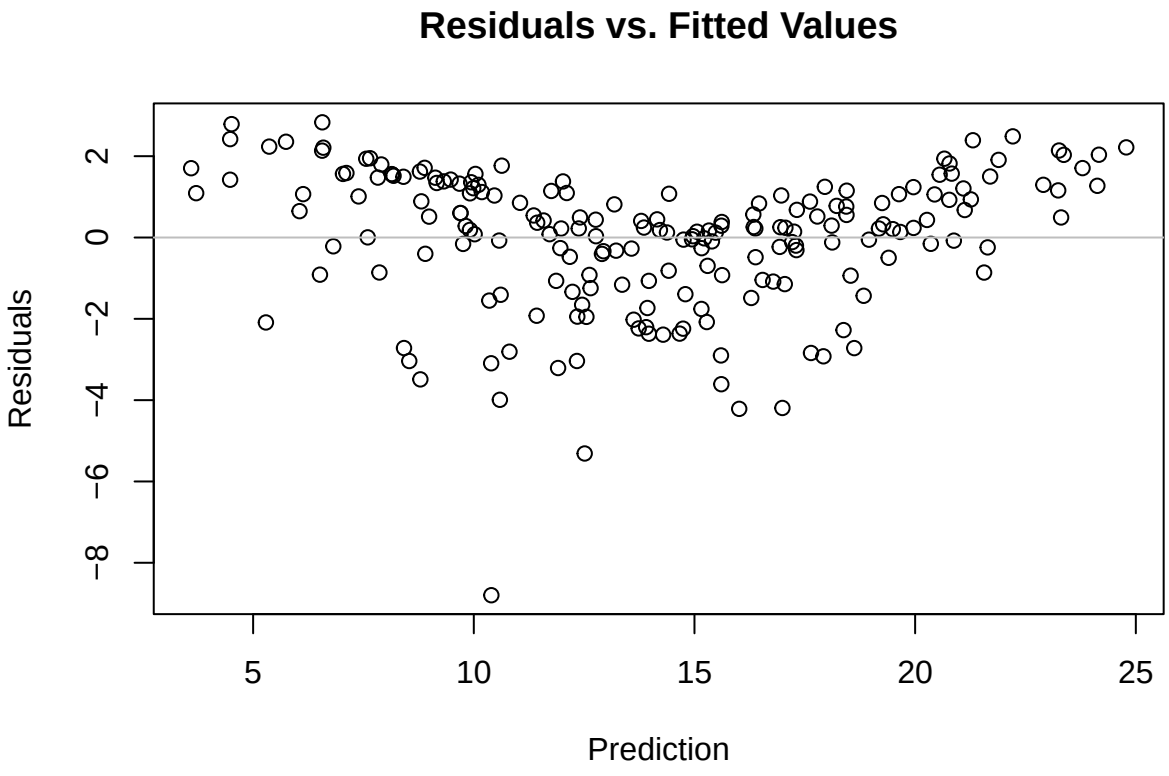

```r
plot(resid(md2) ~ TV, main = "Residuals vs. TV", xlab = "TV", ylab = "Residuals")
abline(h = 0, col = "gray")
```



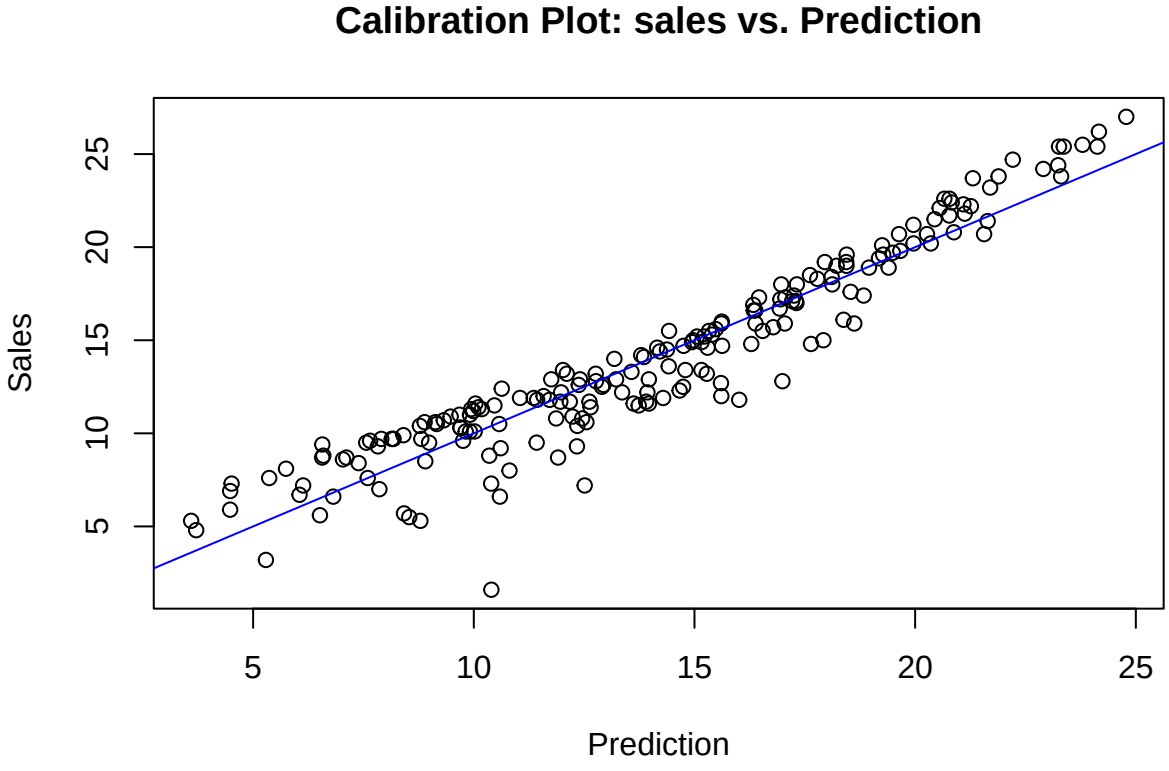
```r
plot(resid(md2) ~ radio, main = "Residuals vs. Radio", xlab = "Radio", ylab =
 ↪  "Residuals")
abline(h = 0, col = "gray")
```



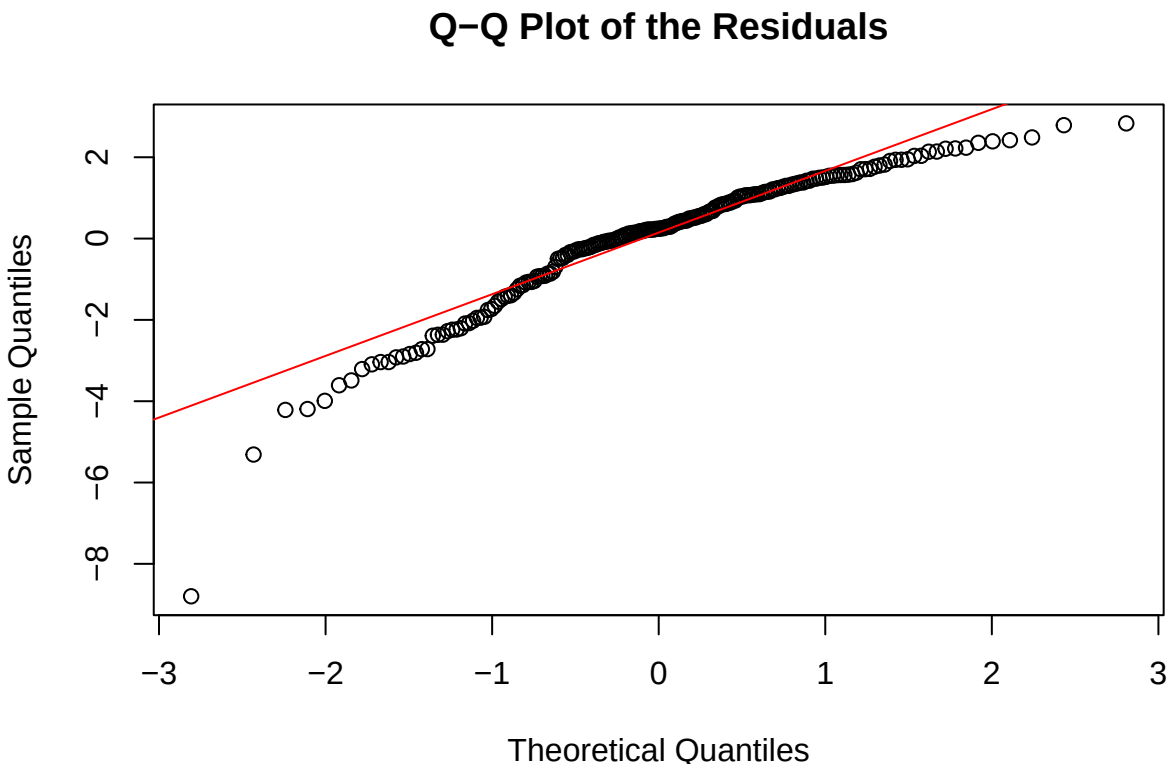

```r
# Residuals vs. fitted values
plot(resid(md2) ~ predict(md2), main = "Residuals vs. Fitted Values", xlab =
 ↪  "Prediction", ylab = "Residuals")
abline(h = 0, col = "gray")
```




```r
# Calibration plot
plot(sales ~ predict(md2), main = "Calibration Plot: sales vs. Prediction", xlab =
 ↪  "Prediction", ylab = "Sales")
abline(0, 1, col = "blue")
```



```r
# QQ plot of the residuals
qqnorm(resid(md2), main = "Q-Q Plot of the Residuals")
qqline(resid(md2), col = "red")
```



Statistical explanation:

- The **residual analysis** serves to check the assumptions of linear regression:
  - Normality: the residuals should be normally distributed.
  - Homoscedasticity: the variance of the residuals should be constant.
  - Independence: the residuals should be independent of one another.
  - Linearity: the relationship between the predictors and the dependent variable should be linear.
- **Residual plots** show the residuals as a function of the predictors or the fitted values.
- A **histogram of the residuals** provides information about the distribution of the residuals.
- A **Q-Q plot** compares the distribution of the residuals with the normal distribution.
- A **calibration plot** shows the observed values as a function of the fitted values. Ideally, the points lie on the diagonal.

R explanation:

- The function `resid()` extracts the residuals from a regression model.
- The function `hist()` creates a histogram.
- The function `plot()` with a formula creates a scatterplot.
- The function `abline()` with the parameter `h` adds a horizontal line to the plot.
- The function `qqnorm()` creates a Q-Q plot that compares the distribution of the residuals with the normal distribution.
- The function `qqline()` adds a reference line to the Q-Q plot.

9 Extension: Multiple Regression with Three Predictors

We now extend the model with a third predictor: Newspaper.

```
# Model with TV, Radio and Newspaper
md3 <- lm(sales ~ TV + radio + newspaper)
summary(md3)
```

```
##
## Call:
## lm(formula = sales ~ TV + radio + newspaper)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.8277 -0.8908  0.2418  1.1893  2.8292
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.938889   0.311908   9.422   <2e-16 ***
## TV           0.045765   0.001395  32.809   <2e-16 ***
## radio        0.188530   0.008611  21.893   <2e-16 ***
## newspaper   -0.001037   0.005871  -0.177     0.86
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.686 on 196 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8956
## F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

```
# Comparison of R² and RSE
cat("Model comparison:\n")
```

```
## Model comparison:
```

```
cat("R² (TV + Radio):", round(summary(md2)$r.squared, 4), "\n")
```

```
## R² (TV + Radio): 0.8972
```

```
cat("R² (TV + Radio + Newspaper):", round(summary(md3)$r.squared, 4), "\n")
```

```
## R² (TV + Radio + Newspaper): 0.8972
```

```
cat("RSE (TV + Radio):", round(summary(md2)$sigma, 4), "\n")
```

```
## RSE (TV + Radio): 1.6814
```

```
cat("RSE (TV + Radio + Newspaper):", round(summary(md3)$sigma, 4), "\n\n")
```

```
## RSE (TV + Radio + Newspaper): 1.6855
```

```
# Adjusted R²
cat("Adjusted R² (TV + Radio):", round(summary(md2)$adj.r.squared, 4), "\n")
```

```
## Adjusted R² (TV + Radio): 0.8962
```

```
cat("Adjusted R² (TV + Radio + Newspaper):", round(summary(md3)$adj.r.squared, 4), "\n")
```

```
## Adjusted R² (TV + Radio + Newspaper): 0.8956
```

Statistical explanation:

- In **multiple regression with three predictors**, the relationship is represented by a hyperplane in four-dimensional space: y = + x + x + x
- The **adjusted R^2** takes the number of predictors in the model into account and penalises models with too many predictors:
  - Adj. $R^2 = 1 - [(1 - R^2) * (n - 1) / (n - p - 1)]$
  - n is the number of observations
  - p is the number of predictors
- The adjusted R^2 can decrease if an added predictor contributes little to the explanatory power of the model.
- The **residual standard error (RSE)** is a measure of the typical size of the residuals and should be smaller for a better model.
- In this example, adding Newspaper leads to a minimal improvement in R^2, but the adjusted R^2 actually decreases slightly, which suggests that Newspaper does not make a substantial contribution to the explanatory power of the model.

R explanation:

- The formula `sales ~ TV + radio + newspaper` in `lm()` specifies that `sales` is the dependent variable and `TV`, `radio` and `newspaper` are the independent variables.
- The expression `summary(md2)$adj.r.squared` extracts the adjusted R^2 from the regression object.
- The expression `summary(md2)$sigma` extracts the RSE from the regression object.

10 Non-linear Models: Quadratic and Polynomial Regression

Not all relationships are linear. We now consider non-linear models, in particular quadratic and polynomial regression.

```
# Quadratic regression
md.q2 <- lm(sales ~ TV + I(TV^2) + radio)
summary(md.q2)
```

```
##
## Call:
## lm(formula = sales ~ TV + I(TV^2) + radio)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.3860 -0.8822 -0.0498  0.9613  3.5725
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.288e+00  3.588e-01   3.588 0.000421 ***
## TV           7.844e-02  4.985e-03  15.736  < 2e-16 ***
## I(TV^2)     -1.136e-04  1.677e-05  -6.775 1.42e-10 ***
## radio        1.930e-01  7.293e-03  26.465  < 2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.517 on 196 degrees of freedom
## Multiple R-squared:  0.9167, Adjusted R-squared:  0.9154
## F-statistic:   719 on 3 and 196 DF,  p-value: < 2.2e-16
```

```
# Polynomial regression (degree 3)
md.q3 <- lm(sales ~ poly(TV, 3) + radio)
summary(md.q3)
```

```
##
## Call:
## lm(formula = sales ~ poly(TV, 3) + radio)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -6.1892 -0.8451 -0.0614  0.7654  3.7264
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    9.464052   0.189302  49.994  < 2e-16 ***
## poly(TV, 3)1  55.323451   1.429492  38.701  < 2e-16 ***
## poly(TV, 3)2 -10.394968   1.434792  -7.245 9.82e-12 ***
## poly(TV, 3)3   7.373480   1.432197   5.148 6.39e-07 ***
## radio          0.195944   0.006884  28.463  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.427 on 195 degrees of freedom
## Multiple R-squared:  0.9267, Adjusted R-squared:  0.9252
## F-statistic:   616 on 4 and 195 DF,  p-value: < 2.2e-16
```

```
# Model comparison: Adjusted R²
cat("Model comparison (non-linear):\n")
```

```
## Model comparison (non-linear):
```

```
cat("R² (linear):", round(summary(md2)$adj.r.squared, 4), "\n")
```

```
## R² (linear): 0.8962
```

```
cat("R² (quadratic):", round(summary(md.q2)$adj.r.squared, 4), "\n")
```

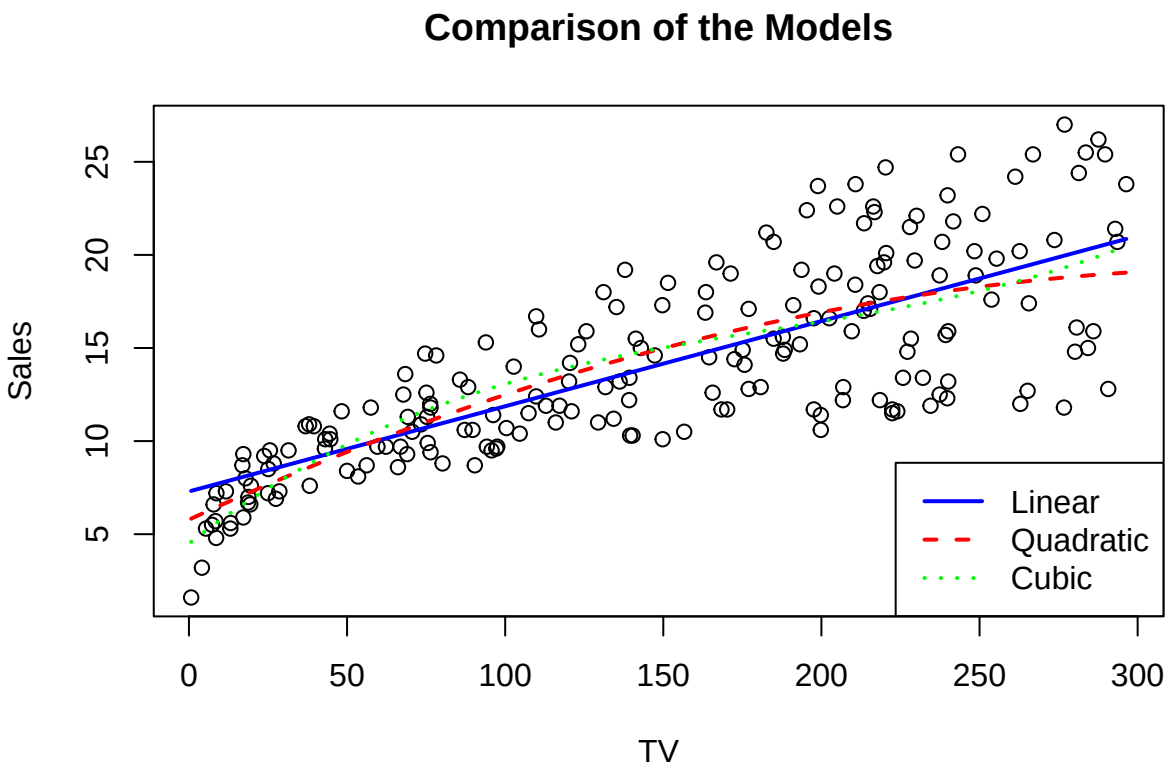
```
## R² (quadratic): 0.9154
```

```
cat("R² (cubic):", round(summary(md.q3)$adj.r.squared, 4), "\n\n")
```

```
## R² (cubic): 0.9252
```

```
# Visualisation of the models
# Build a grid of TV values
```

```r
tv_grid <- seq(min(TV), max(TV), length.out = 100)
radio_mean <- mean(radio)

# Predictions for the various models
pred_linear <- predict(md2, newdata = data.frame(TV = tv_grid, radio = radio_mean))
pred_quad <- predict(md.q2, newdata = data.frame(TV = tv_grid, radio = radio_mean))
pred_poly3 <- predict(md.q3, newdata = data.frame(TV = tv_grid, radio = radio_mean))

# Plot
plot(TV, sales, main = "Comparison of the Models", xlab = "TV", ylab = "Sales")
lines(tv_grid, pred_linear, col = "blue", lwd = 2)
lines(tv_grid, pred_quad, col = "red", lwd = 2, lty = 2)
lines(tv_grid, pred_poly3, col = "green", lwd = 2, lty = 3)
legend("bottomright",
       legend = c("Linear", "Quadratic", "Cubic"),
       col = c("blue", "red", "green"),
       lty = c(1, 2, 3),
       lwd = 2)
```



Statistical explanation:

- **Non-linear regression** models non-linear relationships between the predictors and the dependent variable.
- **Quadratic regression** adds a quadratic term: y = + x + x^2 +
- **Polynomial regression** generalises this to higher powers: y = + x + x^2 + x^3 + … + x +
- The function `I(TV^2)` in R adds a quadratic term while preserving the original scale.
- The function `poly(TV, 3)` adds orthogonal polynomials up to degree 3, which is numerically more stable.
- Non-linear models can capture more complex relationships, but they are also more susceptible to

overfitting.
- The choice between linear and non-linear models should be based on the nature of the problem, the visualisation of the data, and model comparison criteria such as the adjusted R^2.

R explanation:

- The function `I()` in a formula protects the expression from interpretation by the formula language.
- The function `poly()` produces orthogonal polynomials that are numerically more stable than simple powers.
- The function `seq()` produces a sequence of values.
- The function `lines()` adds lines to an existing plot.
- The parameters `col`, `lwd` and `lty` in `lines()` specify the colour, width and type of the line.

11 Model Comparison: R^2, Adjusted R^2 and RSE

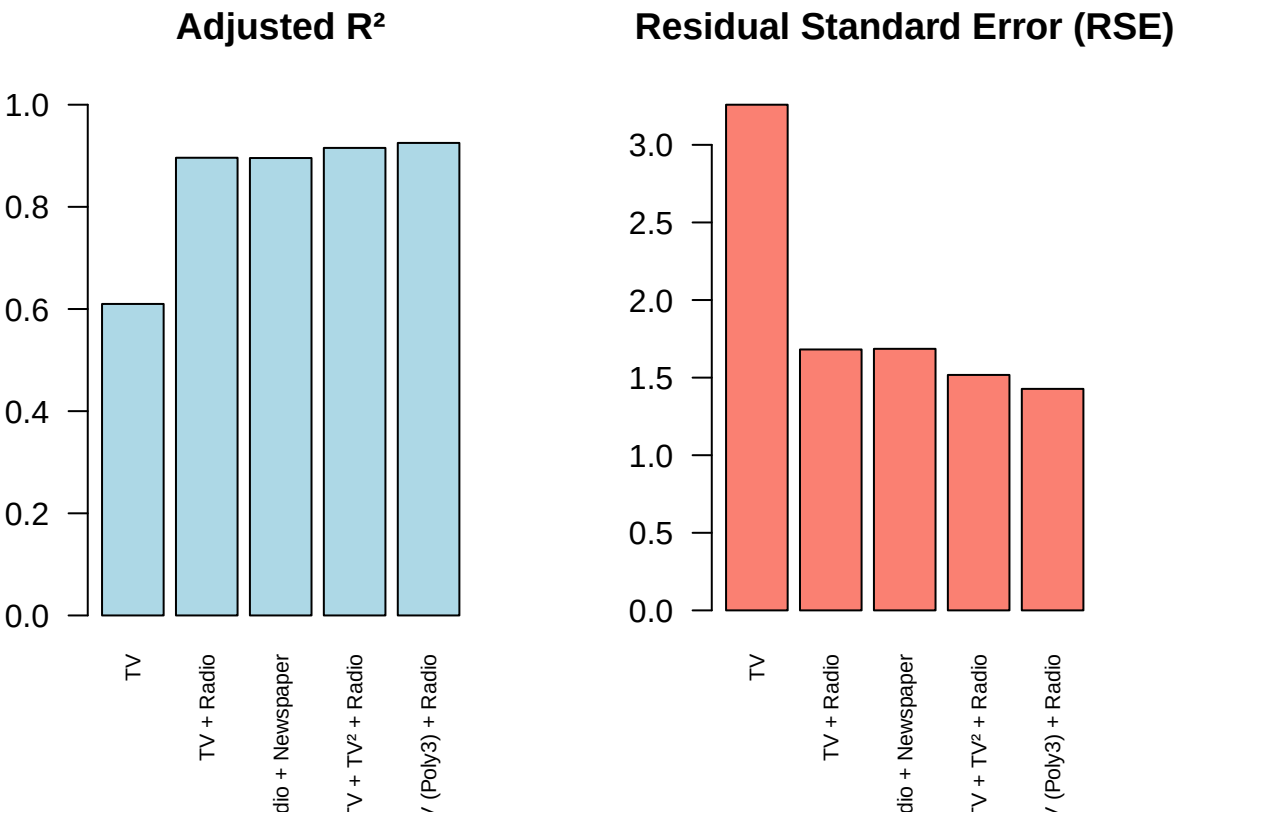
We now systematically compare the various models on the basis of R^2, adjusted R^2 and RSE.

```
# Model comparison: R², Adjusted R² and RSE
models <- list(
  "TV" = md1,
  "TV + Radio" = md2,
  "TV + Radio + Newspaper" = md3,
  "TV + TV² + Radio" = md.q2,
  "TV (Poly3) + Radio" = md.q3
)

# Extract R², Adj. R² and RSE for each model
model_stats <- data.frame(
  Model = names(models),
  R2 = sapply(models, function(m) summary(m)$r.squared),
  Adj_R2 = sapply(models, function(m) summary(m)$adj.r.squared),
  RSE = sapply(models, function(m) summary(m)$sigma)
)

# Output the model statistics
# Round only the numeric columns
model_stats_rounded <- model_stats
model_stats_rounded[, 2:4] <- round(model_stats[, 2:4], 4)
print(model_stats_rounded)
```

```
##                                         Model     R2 Adj_R2    RSE
## TV                                         TV 0.6119 0.6099 3.2587
## TV + Radio                       TV + Radio 0.8972 0.8962 1.6814
## TV + Radio + Newspaper TV + Radio + Newspaper 0.8972 0.8956 1.6855
## TV + TV² + Radio             TV + TV² + Radio 0.9167 0.9154 1.5173
## TV (Poly3) + Radio         TV (Poly3) + Radio 0.9267 0.9252 1.4273
```

```
# Visualisation of the model comparison
par(mfrow = c(1, 2))
barplot(model_stats$Adj_R2, names.arg = model_stats$Model,
        main = "Adjusted R²", las = 2, cex.names = 0.7,
        col = "lightblue", ylim = c(0, 1))
barplot(model_stats$RSE, names.arg = model_stats$Model,
        main = "Residual Standard Error (RSE)", las = 2, cex.names = 0.7,
```

```
        col = "salmon")
```




```
par(mfrow = c(1, 1))
```

Statistical explanation:

- **Model comparison** is an important step in selecting the best model for the data.
- **R^2** gives the proportion of explained variation, but it always increases when predictors are added, even if they are not informative.
- The **adjusted R^2** takes the number of predictors into account and penalises models with too many predictors.
- The **residual standard error (RSE)** gives the typical size of the residuals and should be smaller for a better model.
- In model selection one should find a compromise between goodness of fit and model complexity:
  - A model that is too simple may miss important relationships (underfitting).
  - A model that is too complex may model noise and generalise poorly (overfitting).
- In this example, the model "TV + TV^2 + Radio" appears to offer the best compromise, since it has the highest adjusted R^2 and a low RSE.

R explanation:

- The function `list()` creates a list of objects.
- The function `sapply()` applies a function to each element of a list and returns a vector.
- The function `par(mfrow = c(1, 2))` divides the graphics window into 1 row and 2 columns.
- The function `barplot()` creates a bar chart.
- The parameter `las = 2` in `barplot()` rotates the axis labels.
- The parameter `cex.names = 0.7` in `barplot()` reduces the font size of the names.

12 Summary

In this lecture we have introduced multiple linear regression, an extension of simple linear regression to several predictors. The key concepts were:

1. **Multiple regression with several predictors**: multiple regression models the relationship between a dependent variable and several independent variables. The coefficients give the partial effect of each variable when all other variables are held constant.
2. **Residual analysis and goodness of fit**: the analysis of the residuals helps to check the assumptions of linear regression. Goodness of fit is quantified by measures such as R^2, adjusted R^2 and RSE.
3. **Prediction and error margin**: with the regression model, predictions can be made for new values of the independent variables. Prediction and confidence intervals quantify the uncertainty of these predictions.
4. **Non-linear and polynomial regression**: not all relationships are linear. Quadratic and polynomial regression can model more complex relationships.
5. **Model comparison**: the choice between different models should be based on criteria such as the adjusted R^2 and the RSE, with a compromise between goodness of fit and model complexity.

Multiple regression is a powerful tool for data analysis, but it also has limitations. It assumes linear relationships (unless non-linear terms are added explicitly), can be impaired by multicollinearity, and is susceptible to outliers. In such cases, more robust or more flexible methods may be more appropriate.